Concepts

The vocabulary that comes up in every Claude Code session — skills, MCPs, CLAUDE.md, subagents, plan mode, slash commands.

Through-line from Day 1: **almost all of this is just text.** Skills, commands, CLAUDE.md, even most "tools" are text files Claude reads. Get the right text in front of it and it behaves the way you want.

CLAUDE.md

A plain markdown file that acts as an **automatic, project-level system prompt**. Every time you start a session in a folder, Claude reads the CLAUDE.md at the root (and walks *up* the folder tree, reading any others along the way, up to your home directory). Use it to ground the model in *your* reality — who you are, the kind of work you produce, terms of art in your field, conventions to follow.

It's just text. One sentence ("always end with a limerick") changes behavior for an entire folder of work. Edit, **save**, and **open a new session** for changes to take effect. Generate a starter one with */init*.

Skills

A **reusable, packaged capability** you (or Anthropic) can give Claude — a folder with a SKILL.md (instructions Claude follows) plus any supporting files or scripts. Skills load *on demand*: Claude reads the short description, and when a task matches, it pulls in the full instructions. Saved expertise for recurring jobs ("draft a rubric this way," "format a handout in our house style") so you don't re-explain every time.

MCP (Model Context Protocol)

A **standard way to connect Claude to outside tools and data** — your files, a database, a web browser, an institutional system. An *MCP server* exposes a set of actions Claude can call. Where a skill is *knowledge*, an MCP is usually a *connection* to something live. You add the ones you need; Claude then has new abilities (e.g., "search this database," "control a browser").

Subagents

A **separate Claude working in the background** on a focused task, with its own fresh context window. The main session hands off a job ("research X across these files," "review this draft"), the subagent does the work without cluttering your main conversation, and reports back just the result. Useful for big searches or parallel work — you keep the conclusion, not the mess.

Plan Mode

A working mode where Claude **researches and proposes a plan before changing anything**. Toggle it on (Shift+Tab), describe what you want, and Claude reads, thinks, and comes back with a step-by-step plan for your approval — nothing is written or run until you say go. The safest way to start any non-trivial task, and a natural fit for the idea that *a good plan is itself the deliverable.*

Slash commands

Shortcuts you type to control the session rather than to talk to the model. A command is only recognized at the **start** of a message; anything after it is passed along as arguments. Type / in a session to see what's available.

Commands worth knowing

Notation: *<arg>* = required · *[arg]* = optional. The slash-command set you'll actually reach for.

COMMAND	WHAT IT DOES
/init	Generate a starter **CLAUDE.md** so Claude understands the project. **Run this first in a new folder/codebase.**
/clear	Start a **fresh context window** — drop the current conversation and begin clean. (Cheap, and often the right move when a thread gets long or goes sideways.)
/compact *[instructions]*	Summarize the conversation to **free up context** without losing the thread. Optionally focus the summary.
/context *[all]*	Visualize where your context window is going, with warnings and tips. Pairs with */compact*.
/rewind	Roll **code and/or conversation back** to an earlier checkpoint. (aliases: /checkpoint, /undo)
/resume *[session]*	Return to an earlier conversation by ID/name, or open the picker. (alias: /continue)
/branch *[name]*	**Fork the current conversation,** preserving the original to return to later. (alias: /fork)
/btw *<question>*	Ask a quick side question **without** bloating the main conversation history.
/usage	Session cost, plan limits, and activity stats. (aliases: /cost, /stats)

> **Remember:** a command is only recognized at the *start* of a message. Everything after it becomes the command's argument.

Source: Claude Code command reference — code.claude.com/docs/llms.txt